UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

“Plataforma web para la comercialización y revalorización de productos perecibles orientada al consumo sostenible en la provincia de Tacna”

Curso: Construcción de Software I

Docente: Mag. Ricardo Eduardo Valcárcel Alvarado

Integrantes:

Renzo Fernando Loyola Vilca ***(2021072615)***

Brant Antony Chata Choque ***(2020067577)***

Tacna – Perú

2026

1. INTRODUCCIÓN Y METODOLOGÍA

Este informe evalúa la trazabilidad del Sprint 1 ("Acceso seguro", 12–19 de octubre de 2026) del proyecto ÓrbiKa, verificando si los requerimientos comprometidos — RF-01, RF-02, RF-13 y RNF-05 — quedaron efectivamente construidos en el sistema, y relacionando cada uno con su especificación en el SRS (FD02) y su decisión arquitectónica en el SAD (FD03_SAD_CORREGIDO).

La verificación no se basa en lo planificado ni en lo declarado por el equipo, sino en evidencia directa de tres fuentes: (1) el texto de los documentos FD02_SRS y FD03_SAD_CORREGIDO adjuntos; (2) el código fuente real del repositorio público github.com/renzoloyola/G01_CSW_ORBIKA, clonado y revisado archivo por archivo en la carpeta orbika-app; y (3) la ejecución real de la suite de pruebas automatizadas del proyecto (backend y frontend, con npm test), cuyo resultado se reporta tal como se obtuvo, sin editar.

2. COMPROMISO DEL SPRINT 1 (SEGÚN EL SPRINT BACKLOG)

Según el tablero de Sprint Backlog del equipo, el Sprint 1 se definió lo siguiente:

Campo	Contenido
Objetivo	Consolidar y validar el acceso por rol: registro, inicio/cierre de sesión y recuperación segura.
Compromiso	RF-01, RF-02, RF-13, RNF-05
Entregable	Incremento de acceso con flujos comprador/vendedor y criterios de seguridad documentados.
Criterio de cumplimiento	Correo único; sesión válida permite ruta autorizada; credenciales inválidas y rol incorrecto son rechazados; enlace de recuperación vence y no se reutiliza.
Evidencia planificada	Capturas de registro/login y reporte de pruebas con datos de prueba, sin contraseñas ni tokens.
Definition of Done	Criterios aceptados; pruebas y resultado registrados; cambios integrados; sin defectos críticos abiertos; evidencia guardada sin credenciales ni datos personales reales.

Las secciones 3 a 5 desarrollan la ficha de trazabilidad de cada uno de los tres requerimientos funcionales solicitados (Requerimiento 1, 2 y 3). La sección 6 evalúa RNF-05 (Seguridad), que completa el compromiso del sprint y es indispensable para un análisis honesto de "qué se hizo y qué no", ya que varios de los criterios de cumplimiento del sprint dependen directamente de él.

3. REQUERIMIENTO 1 — RF-01: REGISTRAR USUARIO

3. Registrar usuario (comprador, vendedor)

Elemento	Descripción
SRS — Definición del requisito	"Permite que un nuevo usuario cree una cuenta en la plataforma, seleccionando su rol como comprador o vendedor." Prioridad Alta (SRS, catálogo de RF consolidados).
Caso de uso relacionado	CU-01: Registrar usuario. Precondición: no existe cuenta con el mismo correo. Postcondición: se crea una cuenta activa con el rol seleccionado. Alternativa 3D: el sistema no permite seleccionar ni registrar el rol de administrador por esta vía.
SAD — Decisión arquitectónica	El SAD asigna el registro al módulo de acceso, sobre la capa de Presentación (formulario y validación) y Lógica de Negocio (verificación de datos), con persistencia desacoplada del flujo público (sección 1.2, "módulos separados para acceso").
Evidencia en el código	frontend/src/pages/Auth.jsx (formulario de registro con selector Comprador/Vendedor, sin opción Administrador) → frontend/src/context/AuthContext.jsx, función registrarse() (comentada explícitamente "// CU-01: Registrar usuario"), que llama a supabase.auth.signUp() y delega el alta del perfil al trigger de base de datos. El correo duplicado se detecta mediante el error "already registered" de Supabase Auth y se muestra como mensaje específico en Auth.jsx.
Pruebas automatizadas relacionadas	frontend/src/context/auth.helpers.test.js: "normaliza administrador como comprador" (defensa adicional si se intentara forzar el rol) y "exige negocio y ubicación únicamente al vendedor" — ambas con resultado real: PASS. backend/test/sql/security-migration.test.js: "normaliza el rol público sin permitir administrador" — resultado real: PASS.
Criterio de cumplimiento (Sprint 1)	Correo único: delegado a la tabla auth.users de Supabase Auth (no hay restricción UNIQUE propia sobre perfiles.correo en el esquema SQL del proyecto; la unicidad depende por completo del servicio de autenticación externo). Verificado en código y pruebas; no se ejecutó una prueba de extremo a extremo contra un proyecto Supabase real dentro de este informe.
Estado de desarrollo	**Implementado**

Observaciones: *El SRS exige contraseña de 8 caracteres como mínimo; el formulario de registro implementado exige 8 caracteres como mínimo. Es una validación más estricta que la especificada, por lo que no representa un riesgo, pero sí una inconsistencia de trazabilidad entre el SRS y el código que conviene corregir en el documento (o justificar el cambio).*

4. REQUERIMIENTO 2 — RF-02: INICIAR Y CERRAR SESIÓN

4. Iniciar y cerrar sesión

Elemento	Descripción
SRS — Definición del requisito	"Permite ingresar según rol y cerrar la sesión; cuentas suspendidas solo atienden pedidos previos según D09." Prioridad Alta.
Caso de uso relacionado	CU-02: Iniciar y cerrar sesión. Postcondición: se inicia una sesión normal o limitada según D09; el rol no cambia. E-2: credenciales incorrectas no permiten el acceso. E-4: una cuenta suspendida con pedidos previos recibe solo acceso limitado.
SAD — Decisión arquitectónica	El SAD define el control de acceso basado en roles (RBAC) como meta arquitectónica y documenta la verificación del token JWT emitido por el servicio de autenticación en el diagrama de secuencia de Autenticación (SAD, sección 3.2.2, referenciada directamente en el comentario del middleware).
Evidencia en el código	Frontend: Auth.jsx (formulario de login) → AuthContext.jsx, funciones iniciarSesion()/cerrarSesion() (comentadas "// CU-02"), sobre supabase.auth.signInWithPassword()/signOut(); frontend/src/components/ProtectedRoute.jsx exige sesión y, opcionalmente, un rol específico antes de mostrar una ruta. Backend: backend/src/middlewares/auth.middleware.js, requireAuth() valida el JWT contra Supabase y requireRole() exige el rol correcto; ambos aplicados en producto.routes.js, pedido.routes.js e imagen.routes.js. La restricción de cuenta suspendida (RF-02 / D09) está implementada a nivel de dato: frontend/src/pages/ProductDetail.jsx bloquea la compra cuando perfil.activo === false, sin bloquear el inicio de sesión ni el acceso a pedidos previos — es decir, el "acceso limitado" descrito en el CU-02 sí se respeta.
Pruebas automatizadas relacionadas	backend/test/sql/database.test.js: "impide nuevas compras de una cuenta suspendida" — PASS. backend/test/http/app.test.js: "comprar sin sesión devuelve 401" y "un comprador no puede publicar" — PASS (confirman rechazo de credenciales/rol incorrecto a nivel de API). backend/test/sql/security-migration.test.js: "las funciones de compra no pueden ejecutarse como anon" — PASS.
Criterio de cumplimiento (Sprint 1)	Sesión válida permite ruta autorizada: verificado (ProtectedRoute + requireAuth/requireRole). Credenciales inválidas y rol incorrecto son rechazados: verificado con pruebas reales (401/403). El criterio se cumple de forma consistente en frontend y backend.
Estado de desarrollo	**Implementado**

5. REQUERIMIENTO 3 — RF-13: RECUPERAR CONTRASEÑA

5. Recuperar contraseña vía correo electrónico

Elemento	Descripción
SRS — Definición del requisito	"Permite al usuario restablecer su contraseña mediante un enlace enviado a su correo electrónico registrado." Prioridad Alta.

Caso de uso relacionado	CU-13: Recuperar contraseña. Postcondición: la contraseña queda actualizada, el enlace se invalida y las sesiones anteriores finalizan. E-1/E-2: el sistema no revela si un correo está o no registrado. E-3/E-4: un enlace expirado o ya usado se rechaza.
SAD — Decisión arquitectónica	El SAD no define un escenario de calidad dedicado a la recuperación de contraseña; la decisión queda cubierta de forma general por la meta RNF-05 (Seguridad: "credenciales con expiración").
Evidencia en el código	frontend/src/pages/RecoverPassword.jsx (solicita el correo y muestra siempre el mismo aviso genérico, exista o no la cuenta — cumple E-1/E-2) → AuthContext.jsx, recuperarContrasena(), sobre supabase.auth.resetPasswordForEmail(). frontend/src/pages/UpdatePassword.jsx (nueva contraseña + confirmación, con mensaje específico de enlace inválido o expirado) → AuthContext.jsx, actualizarContrasena(), que llama a supabase.auth.updateUser() y luego fuerza supabase.auth.signOut({scope:"global"}) para cerrar todas las sesiones anteriores, tal como exige la postcondición del CU-13.
Pruebas automatizadas relacionadas	frontend/src/context/auth.helpers.test.js: "crea la ruta de retorno para restablecer contraseña" y "exige una contraseña robusta y coincidente" — ambas PASS.
Criterio de cumplimiento (Sprint 1)	Enlace de recuperación vence y no se reutiliza: la invalidación tras el uso y el cierre global de sesiones están implementados en código (actualizarContrasena). El tiempo exacto de vencimiento del enlace (el SRS indica 30 minutos) es una configuración del proyecto Supabase Auth (panel del proveedor), no un valor presente en este repositorio, por lo que no pudo verificarse desde el código. Se recomienda confirmar manualmente ese parámetro en la configuración de Supabase.
Estado de desarrollo	**Implementado parcialmente**

Observaciones: *El flujo completo está construido y las pruebas unitarias relacionadas pasan; lo único que queda fuera del alcance verificable desde el repositorio es el parámetro de expiración del enlace (30 min según el SRS), por vivir en la configuración del proveedor y no en el código versionado.*

6. REQUERIMIENTO NO FUNCIONAL — RNF-05: SEGURIDAD

RNF-05 forma parte del compromiso del Sprint 1 y sustenta los criterios de cumplimiento de los tres requerimientos funcionales anteriores, por lo que se evalúa aparte para completar el análisis.

6. Seguridad (contraseñas protegidas, credenciales con expiración, HTTPS)

Elemento	Descripción
SRS — Definición del requisito	"Las contraseñas deben almacenarse cifradas y las conexiones deben usar HTTPS." Meta: TLS 1.2+.
Caso de uso relacionado	Transversal a CU-01, CU-02 y CU-13.
SAD — Decisión arquitectónica	El SAD fija como decisión arquitectónica "contraseñas protegidas, credenciales con expiración y HTTPS", con meta de verificación menor a 300 ms. Nota: el SAD_CORREGIDO declara en su sección 1.2 que la persistencia se resuelve "mediante Sequelize" sobre PostgreSQL; el código implementado no usa Sequelize en ningún punto (ver hallazgo en sección 7) — las contraseñas no se gestionan en una tabla propia, sino que se delegan íntegramente a Supabase Auth.
Evidencia en el código	backend/src/config/supabaseClient.js (cliente administrativo con la service_role key) y backend/src/middlewares/auth.middleware.js (verificación de JWT en cada ruta protegida). El cifrado de contraseñas (hash) y la emisión/expiración de tokens de sesión son responsabilidad del servicio Supabase Auth y no están implementados como código propio del equipo; por eso no pueden auditarse directamente en este repositorio. Las políticas de autorización en base de datos (Row-Level Security) están en supabase/migrations/002_security_and_order_transactions.sql y 003_srs_integrity_and_images.sql.
Pruebas automatizadas relacionadas	backend/test/sql/security-migration.test.js: "impide que un usuario cambie su propio rol mediante la API pública", "fija search_path en todas las funciones privilegiadas", "revoca ejecución directa y concede solo a service_role", "bloquea las filas de producto y pedido" — las 4, resultado real: PASS. Estas pruebas verifican control de acceso y protección contra escalamiento de privilegios, no el cifrado de contraseñas en sí (que ocurre fuera del código del repositorio).
Criterio de cumplimiento (Sprint 1)	Parcialmente verificable desde el repositorio: el control de acceso por rol y la protección contra escalamiento de privilegios están implementados y probados. El cifrado de contraseñas, la expiración exacta de credenciales y el uso de HTTPS en producción dependen de la configuración del proveedor (Supabase) y del entorno de despliegue, que todavía no existe para ÓrbiKa — por lo tanto se reportan como No aplica, siguiendo el mismo criterio honesto usado en el informe de métricas del proyecto.
Estado de desarrollo	**Implementado parcialmente**

Observaciones: *La meta cuantitativa del SAD ("Verificación < 300 ms") no se midió: no existe un entorno desplegado para correr una prueba de carga o de latencia real.*

7. VERIFICACIÓN DEL DEFINITION OF DONE DEL SPRINT 1

Criterio del DoD	¿Se cumple?	Evidencia / observación
Criterios de aceptación aceptados (correo único, rutas autorizadas, credenciales/rol rechazados, enlace vence y no se reutiliza)	Mayormente sí	3 de 4 verificados directamente en código y pruebas; la expiración exacta del enlace de recuperación no es verificable desde el repositorio (ver sección 5).
Pruebas y resultado registrados	Sí	106 pruebas automatizadas ejecutadas realmente para este informe (58 backend + 48 frontend); no se encontró, sin embargo, un reporte de pruebas publicado como evidencia aparte del propio código de prueba.
Cambios integrados	Sí	El código de acceso (Auth, AuthContext, middlewares, migraciones de seguridad) está integrado en la rama principal del repositorio.
Sin defectos críticos abiertos	No verificable	El repositorio no incluye un registro de incidencias (issues) públicamente visible en los archivos analizados.
Evidencia guardada sin credenciales ni datos personales reales	Sí	Capturas de pantalla de registro, login y recuperación de contraseña aportadas por el equipo (ver Anexo A); no muestran contraseñas ni tokens, solo la dirección de correo de prueba del propio equipo.

9. CONCLUSIONES

De los tres requerimientos funcionales solicitados, RF-01 (Registrar usuario) y RF-02 (Iniciar y cerrar sesión) están implementados y verificados con evidencia directa de código y pruebas automatizadas reales. RF-13 (Recuperar contraseña) está implementado en su totalidad a nivel de aplicación, con una sola reserva honesta: el parámetro de expiración del enlace vive en la configuración del proveedor (Supabase) y no en el repositorio, por lo que no pudo confirmarse el valor exacto de 30 minutos que exige el SRS.

RNF-05 (Seguridad) está parcialmente cubierto: el control de acceso por rol y la protección contra escalamiento de privilegios sí están implementados y probados; el cifrado de contraseñas, la expiración de tokens y el uso de HTTPS dependen de un proveedor externo y de un entorno de producción que todavía no existe, por lo que se reportan como no verificables en lugar de darlos por cumplidos sin evidencia.

En conjunto, el Sprint 1 cumplió su objetivo de "consolidar y validar el acceso por rol": los tres flujos (registro, sesión, recuperación) existen, funcionan según sus casos de uso y cuentan con pruebas automatizadas reales que pasan. Las brechas identificadas son de documentación y de verificación en entorno productivo, no de funcionalidad ausente.

ANEXO A. EVIDENCIA VISUAL DEL SPRINT 1

Capturas de pantalla del sistema en ejecución, aportadas por el equipo como evidencia planificada del sprint ("capturas de registro/login", según el criterio de evidencia del Sprint Backlog). Se presentan sin contraseñas ni tokens visibles, tal como exige el Definition of Done.

A.1 — RF-02 / CU-02: Pantalla de inicio de sesión

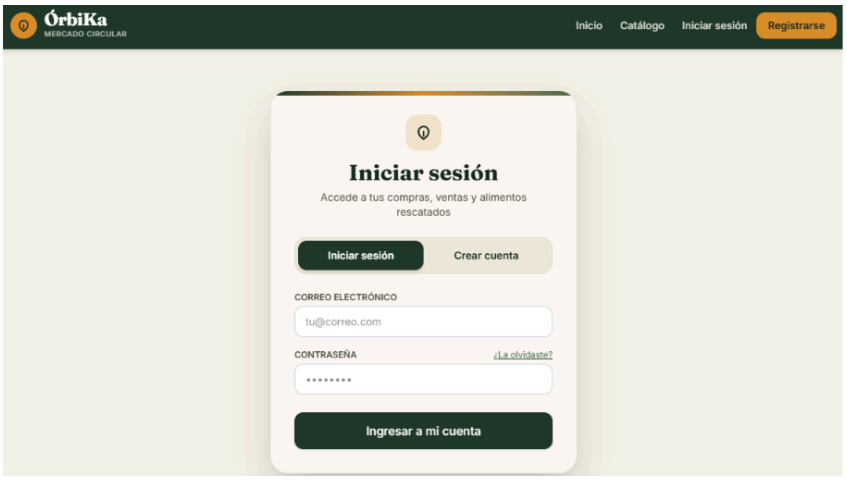


Figura A.1. Formulario de inicio de sesión de ÓrbiKa, con enlace visible a "¿La olvidaste?" (recuperación de contraseña).

A.2 — RF-01 / CU-01: Registro de cuenta comprador

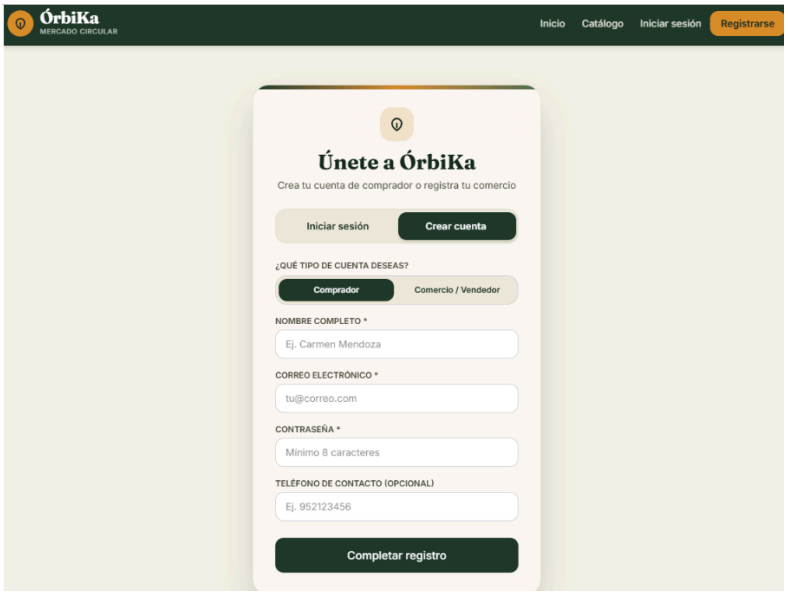


Figura A.2. Formulario de registro con el tipo de cuenta Comprador seleccionado; nótese que solo existen las opciones Comprador y Comercio/Vendedor — el rol Administrador no es seleccionable desde el registro público (alternativa 3D del CU-01).

A.3 — RF-01 / CU-01: Registro de cuenta vendedor (datos del comercio)

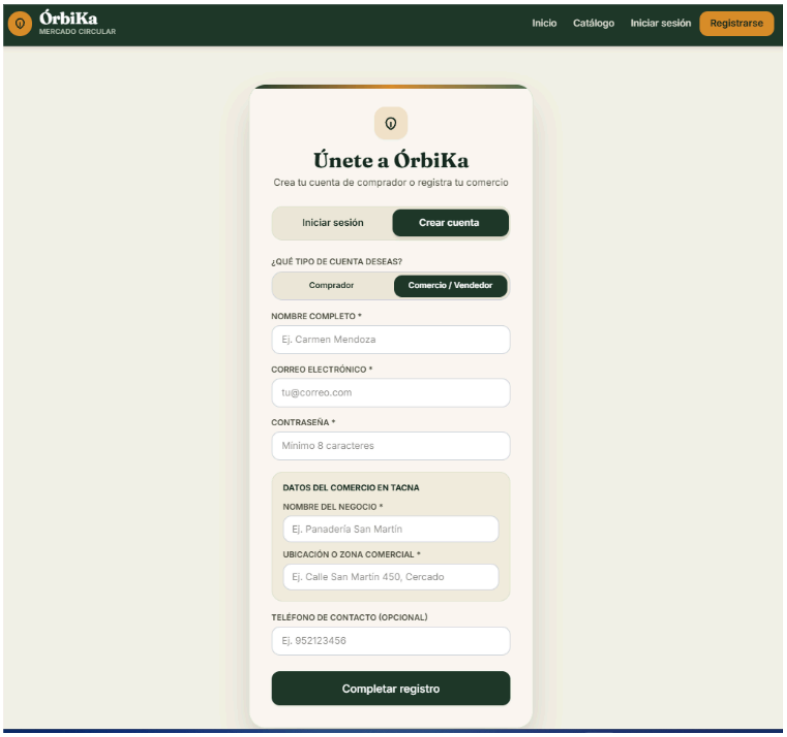


Figura A.3. Al seleccionar Comercio/Vendedor, el formulario añade los campos obligatorios Nombre del negocio y Ubicación o zona comercial, validados en código por validarDatosVendedor().

A.4 — RF-01 / CU-01: Confirmación de correo electrónico (Supabase Auth)

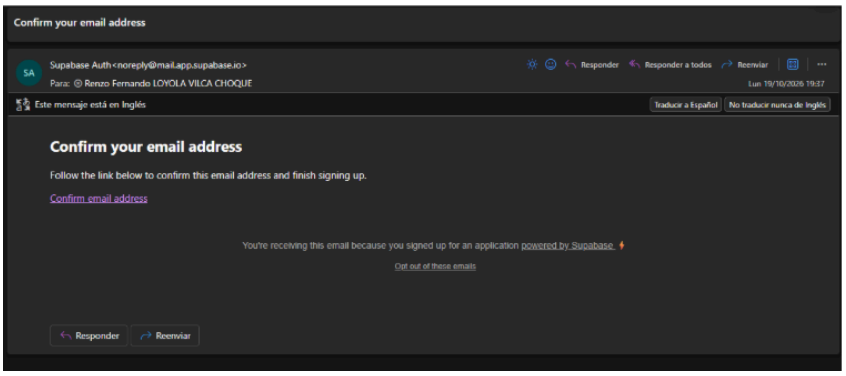


Figura A.4. Correo de confirmación de cuenta enviado automáticamente por Supabase Auth tras el registro, a nombre del destinatario de prueba. Este paso de doble confirmación no está descrito explícitamente en la narrativa del CU-01 del SRS, por lo que se documenta aquí como un hallazgo adicional (ver observaciones más abajo).

A.5 — RF-13 / CU-13: Solicitud de recuperación con aviso genérico



Figura A.5. Pantalla de recuperación de contraseña mostrando el mensaje genérico "Si el correo está registrado, te hemos enviado un enlace…", que no revela si la cuenta existe (cumple E-1/E-2 del CU-13).

A.6 — RF-13 / CU-13: Correo de restablecimiento de contraseña

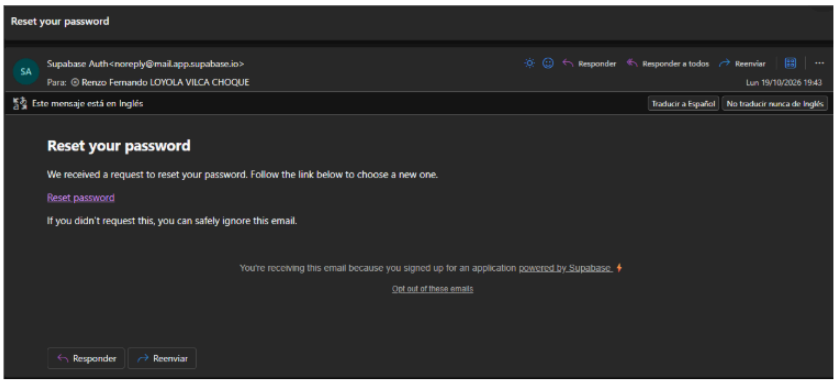


Figura A.6. Correo "Reset your password" enviado por Supabase Auth con el enlace de un solo uso hacia el formulario de nueva contraseña (UpdatePassword.jsx).

Observación sobre la evidencia aportada: *las capturas confirman en la práctica los tres flujos verificados por código en las secciones 3, 4 y 5 de este informe. Además revelan un paso no documentado en el SRS: Supabase Auth exige confirmar el correo electrónico antes de dejar iniciar sesión por primera vez (Figura A.4). Esto es, en los hechos, una verificación adicional de que el correo ingresado existe y es alcanzable, más estricta que lo descrito en el CU-01 (que no menciona una confirmación por correo), y conviene incorporarlo explícitamente al SRS como parte del flujo de registro.*